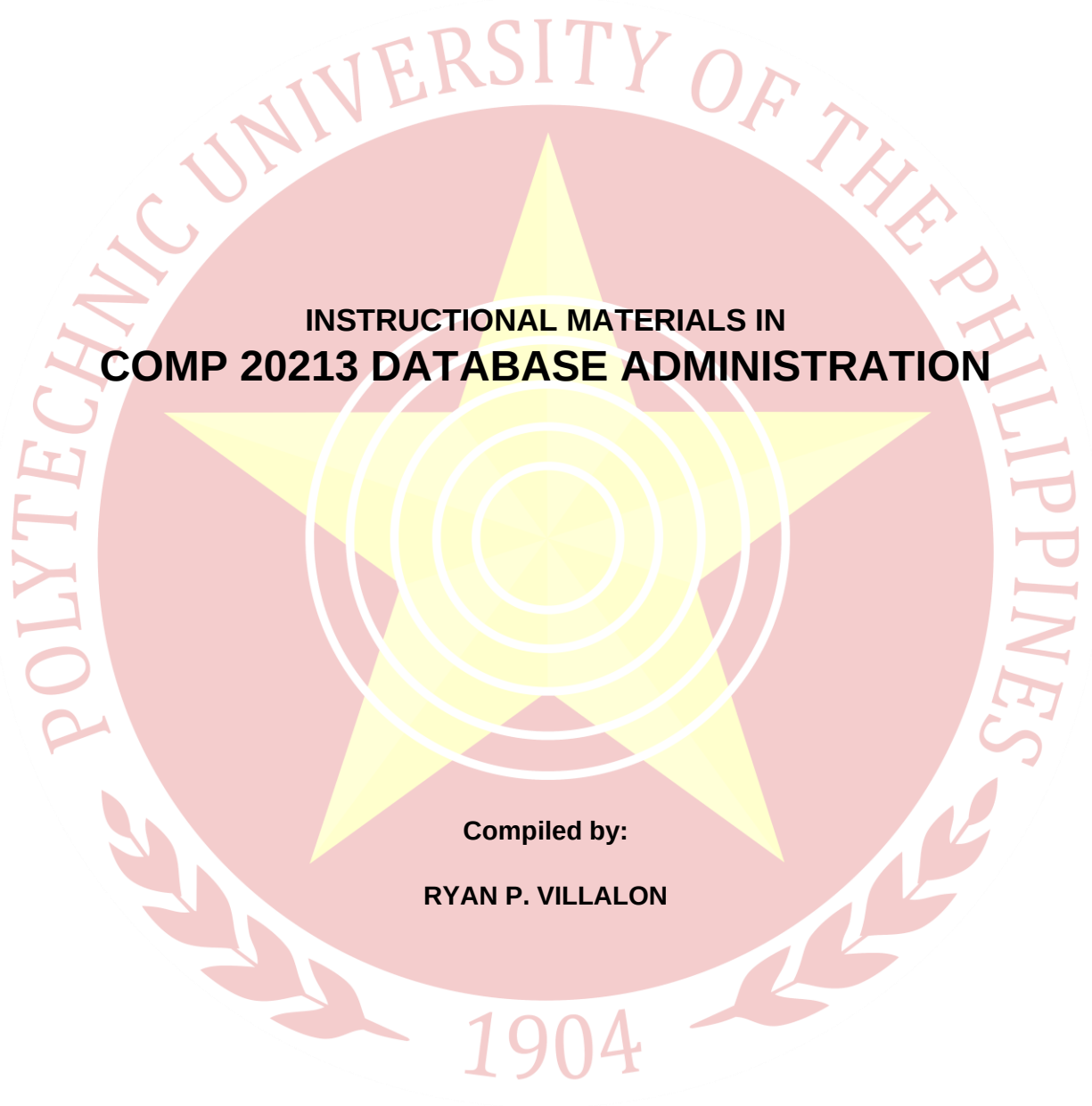




**INSTRUCTIONAL MATERIALS IN
COMP 20213 DATABASE ADMINISTRATION**

Compiled by:

RYAN P. VILLALON



OVERVIEW

This course discusses basic concepts on database administration. It explores the features of technology specific database packages such as IBM DB2 UDB, Microsoft SQL Server, MySQL, Sybase, and Oracle among others. It includes discussions on installation, creation and configuration of databases, configure security, and transfer data between sources/servers, backing up and restoring databases, database concurrency, and optimizing database performance.



TABLE OF CONTENTS

OVERVIEW.....	ii
TABLE OF CONTENTS.....	iii
LESSON OUTCOMES	iv
LESSON 1 INTRODUCTION TO DATABASE ADMINISTRATION	5
LESSON 2 FUNDAMENTALS.....	7
LESSON 3 DESIGNING AND IMPLEMENTING DATABASE.....	12
LESSON 4 THE OVERVIEW OF STRUCTURE QUERY LANGUAGE	18
LESSON 5 DATA MANIPULATING LANGUAGE.....	25
LESSON 6 ADVANCED SQL.....	29
LESSON 7 SUBQUERIES.....	48
LESSON 8 SQL VIEWS	52
LESSON 9 STORE PROCEDURE.....	54
LESSON 10 TRIGGERS.....	57
GRADING SYSTEM	59
REFERENCES	59
APPENDIX.....	60



COURSE OUTCOMES

At the end of the course, students will be able to:

1. Discuss the concept of data and database administrations. Explain the role of database administrator in the management of databases, pertaining to the security, availability, quality of databases.
2. Identify specific SQL clauses that should be used in code development to be able to generate the required outputs.
3. Evaluate existing database management platforms which will suit the current and future needs of an organization.
4. Implement a robust application using a specific DBMS which has been tested against the requirements of a particular business.
5. Apply test scenarios to make sure application works as expected; Ensure seamless implementation of DBMS programs /applications for the business or organization.
6. Identify all specific activities in each phase of a project plan from the planning to the implementation. Incorporate into the plan the scheduling of resources such as time, budget, and people which should make up the project team.
7. Design an application code in the most efficient manner using a particular database management system.
8. Apply the features of the DBMS (back-up, file transfers, security protocols, etc.) to maximize the benefits for application being developed with the end users in mind.
9. Create optimal running programs and systems using SQL and specific database management system, through efficient coding schemes and effective table designs.



LESSON 1 INTRODUCTION TO DATABASE ADMINISTRATION

Learning Outcomes

At the end of the lesson, students will be able to:

1. Define what Data Administration is.
2. Enumerate the DBA Role

Course Materials

Data Administration

The database and the DBMS (Database Management System) are corporate resources that must be managed like any other resource. Data and database administration are the roles generally associated with the management and control of a DBMS and its data. The **Data Administrator (DA)** is responsible for the management of the data resource including **database planning, development and maintenance of standards, policies and procedures, and conceptual/logical data-base design**. The DA consults with and advises senior managers, ensuring that the direction of database development will ultimately support corporate objectives. The **Database Administrator (DBA)** is responsible for the physical realization of the database, including **physical database design and implementation, security and integrity control, maintenance of the operational system**, and ensuring satisfactory performance of the applications for users. The role of the DBA is more technically oriented than the role of the DA, requiring detailed knowledge of the target DBMS and the system environment. In some organizations **there is no distinction between these two roles**; in others, the importance of the corporate resources is reflected in the allocation of teams of staff dedicated to each of these roles.

Database Administrator

A **database administrator (DBA)** is a specialized computer systems administrator who maintains a successful database environment by directing or performing all related activities to keep the data secure. The top responsibility of a DBA professional is to maintain data integrity. This means the DBA will ensure that data is secure from unauthorized access but is available to users.

The DBA has the following major roles:

1. **Data Security** - A DBA needs to know potential weaknesses of the database software and the company's overall system and work to minimize risks. No system is one hundred per cent immune to attacks but implementing best practices can minimize risks.

In the case of a security breach or irregularity, the DBA can consult audit logs to see who has done what to the data. Audit trails are also important when working with regulated data.

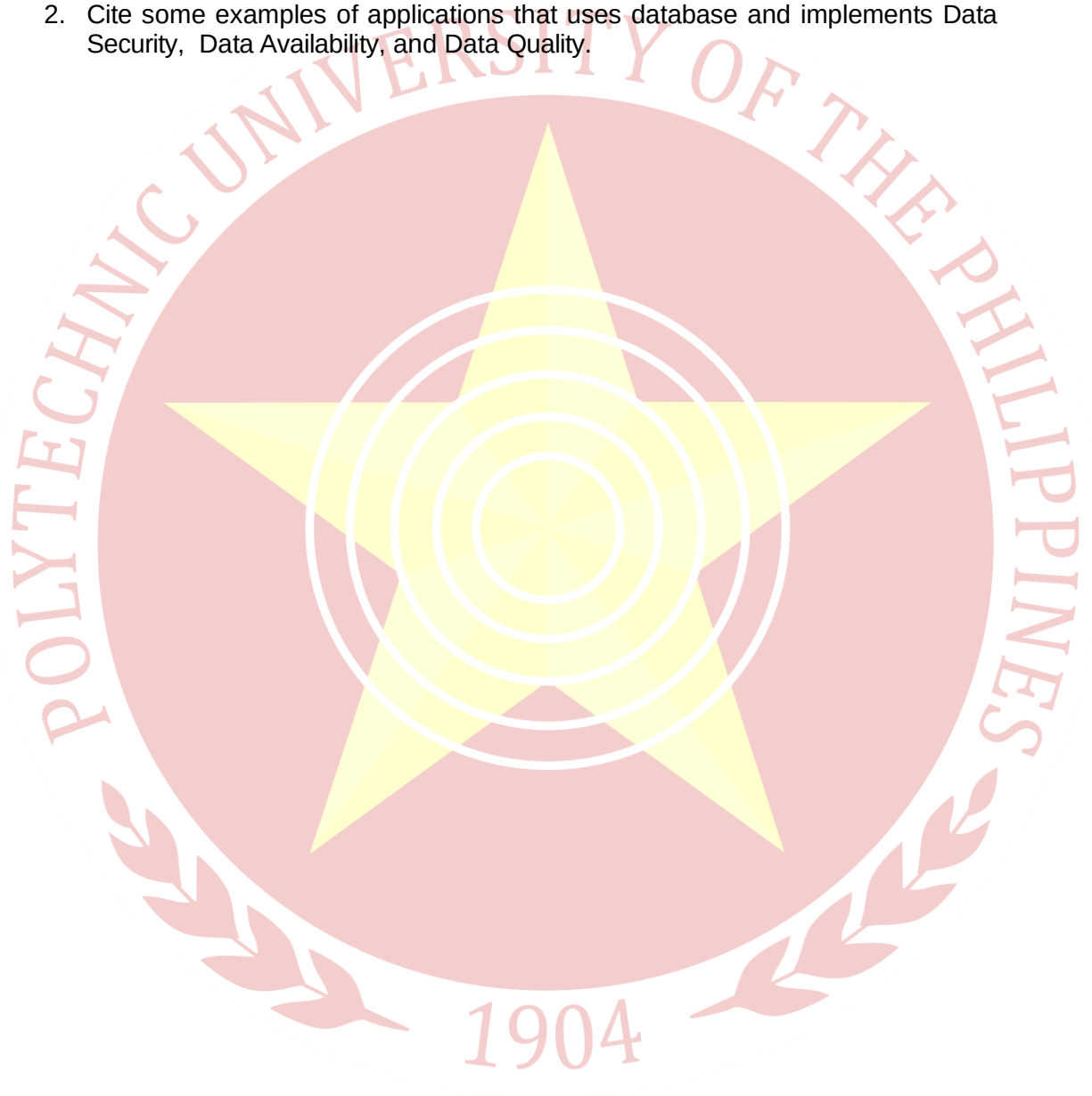
2. **Data Availability** – In relation with data security the DBA must also know how to address the need to access to the database, In general, “who has access, and at what level”. Making sure that data is only available to those authorized to have access to it, and how they can manipulate/modify the data they have access to.
3. **Data Quality** – The DBA must also know how to present the data to its end users,



and how accurate the data is. The DBA must implement steps that only valid data can be inserted in the database and only valid data will be obtained.

Activities/Assessment

1. Briefly explain the difference DBA and DA in an essay format in not less than 150
2. Cite some examples of applications that uses database and implements Data Security, Data Availability, and Data Quality.



LESSON 2 FUNDAMENTALS

Learning Outcomes

At the end of the lesson, students will be able to:

1. Have a thorough understanding of the history of database
2. Differentiate the basic concepts and definition
3. Identify the different database approach
4. Identify the components of database environment

Course Materials

Introduction

Databases are used to store, manipulate, retrieve data of the organization, business, healthcare, education government, libraries, and other institutions. With the rapid growth of data and data capturing mechanisms it essential for an individual to have a thorough knowledge on database management and administrations. This lesson focuses on the history of database, basic definitions of the concepts that include data, information and metadata, data models, entity, relationships, and components of database

History of Database Systems

1950s and early 1960s: Magnetic tapes were developed for data storage. Data processing tasks such as payroll were automated, with data stored on tapes. Processing of data consisted of reading data from one or more tapes and writing data to a new tape. Data could also be input from punched card decks, and output to printers.

Late 1960s and 1970s: Widespread use of hard disks in the late 1960s changed the scenario for data processing greatly, since hard disks allowed direct access to data. The position of data on disk was immaterial, since any location on disk could be accessed in just tens of milliseconds. Data were thus freed from the tyranny of sequentially. With disks, network and hierarchical databases could be created that allowed data structures such as lists and trees to be stored on disk. Programmers could construct and manipulate these data structures. A landmark paper by Codd [1970] defined the relational model and nonprocedural ways of querying data in the relational model, and relational databases were born.

1980s: System R, a groundbreaking project at IBM Research that developed techniques for the construction of an efficient relational database system. The fully functional System R prototype led to IBM's first relational database product, SQL/DS. At the same time, the Ingres system was being developed at the University of California at Berkeley. It led to a commercial product of the same name. Initial commercial relational database systems, such as IBM DB2, Oracle, Ingres, and DEC Rdb, played a major role in advancing techniques for efficient processing of declarative queries. By the early 1980s, relational databases had become competitive with network and hierarchical database systems even in the area of performance.

Early 1990s: The SQL language was designed primarily for decision support applications, which are query-intensive, yet the mainstay of databases in the 1980s was transaction-processing



applications, which are update intensive. Decision support and querying re-emerged as a major application area for databases. Tools for analyzing large amounts of data saw large growths in usage. Many database vendors introduced parallel database products in this period. Database vendors also began to add object-relational support to their databases.

1990s: The major event of the 1990s was the explosive growth of the World Wide Web. Databases were deployed much more extensively than ever before. Database systems now had to support very high transaction-processing rates, as well as very high reliability and 24×7 availability (availability 24 hours a day, 7 days a week, meaning no downtime for scheduled maintenance activities). Database systems also had to support Web interfaces to data.

2000s: The first half of the 2000s saw the emerging of XML and the associated query language XQuery as a new database technology. Although XML is widely used for data exchange, as well as for storing certain complex datatypes, relational databases still form the core of a vast majority of large-scale database applications. In this time period we have also witnessed the growth in “autonomic computing/auto-admin” techniques for minimizing system administration effort. This period also saw a significant growth in use of open-source database systems, particularly PostgreSQL and MySQL.

Basic Concepts and Definitions

- Data** is a raw, unprocessed collection of facts, figures and events that could be stored in a computer media.
- Information** are data that have processed and came from analyzing the data.

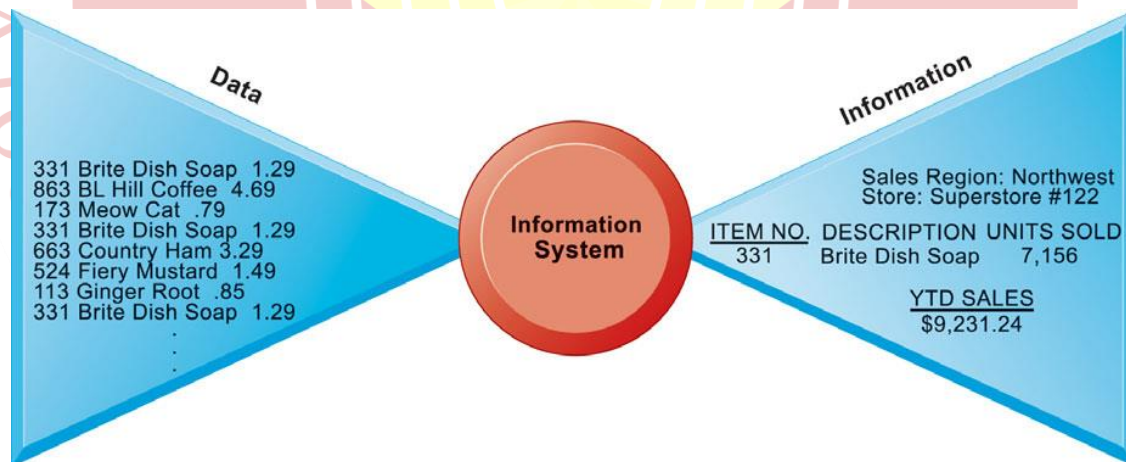


Figure 1.1 Difference between data and information

- Metadata** are data that describe the properties or characteristics of end-user data and the context of that data. Some of the properties that are typically described include data names, definitions, length (or size), and allowable values. It is sometimes referred to as “data about data”. Below is an example of metadata.

Data Item		Metadata				
Name	Type	Length	Min	Max	Description	Source
Course	Alphanumeric	30			Course ID and name	Academic Unit
Section	Integer	1	1	9	Section number	Registrar
Semester	Alphanumeric	10			Semester and year	Registrar
Name	Alphanumeric	30			Student name	Student IS
ID	Integer	9			Student ID (SSN)	Student IS
Major	Alphanumeric	4			Student major	Student IS
GPA	Decimal	3	0.0	4.0	Student grade point average	Academic Unit

Figure 1.2 Database Management System

A database-management system (DBMS) is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the database, contains information relevant to an enterprise. The primary goal of a DBMS is to provide a way to store and retrieve database information that is both convenient and efficient.

Unit 3. The Database Approach

- Data Models** capture the nature of and relationships among data and are used at different levels of abstraction as a database is conceptualized and designed. The effectiveness and efficiency of a database is directly associated with the structure of the database.
- Entity** A person, a place, an object, an event, or a concept in the user environment about which the organization wishes to maintain data. **Attributes** is a property or characteristic of an entity type that is of interest to the organization. An example of entity is a student and all that describes the students are called attribute such as student id, first name, middle name last name, home address and phone numbers.
- Relationships** a well-structured database establishes the relationships between entities that exist in organizational data so that desired information can be retrieved. Most relationships are one-to-many (1:M) or many-to-many (M:N).
- Relational Database** establish the relationships between entities by means of common fields included in a file, called a relation. Below is an example of Relational Database.

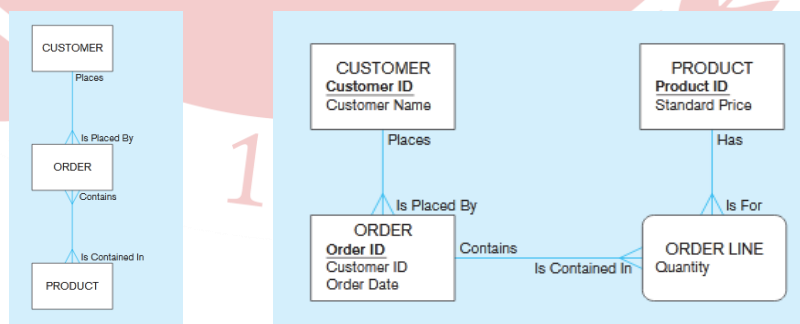


Figure 1.3 Relational Diagram

Exploring the Components

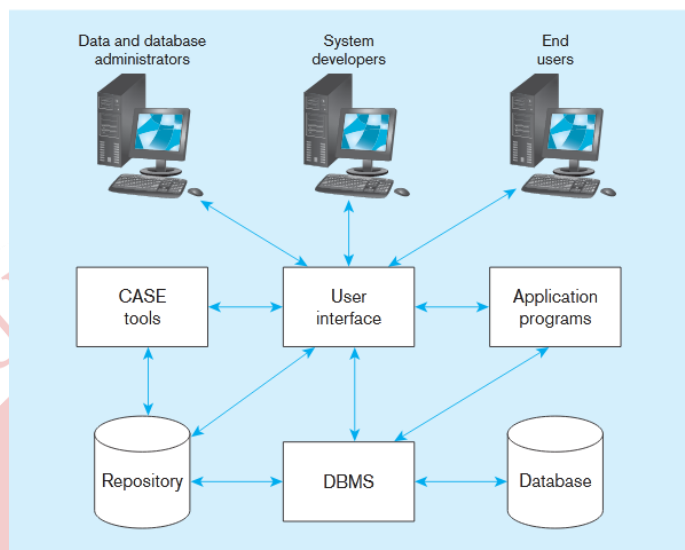


Figure 1.4 Components DBMS

1. **Computer-aided software engineering (CASE) tools** are automated tools used to design databases and application programs. These tools help with creation of data models and in some cases can also help automatically generate the “code” needed to create the database.
2. **Repository** is a centralized knowledge base for all data definitions, data relationships, screen and report formats, and other system components.
3. **DBMS** is a software system that is used to create, maintain, and provide controlled access to user databases.
4. **Database** is an organized collection of logically related data, usually designed to meet the information needs of multiple users in an organization. It is important to distinguish between the database and the repository. The repository contains definitions of data, whereas the database contains occurrences of data.
5. **Application programs** are computer-based application programs are used to create and maintain the database and provide information to users.
6. **User Interface** includes languages, menus, and other facilities by which users interact with various system components, such as CASE tools, applications programs, the DBMS, and the repository.
7. **Data and Database Administrators** are persons who are responsible for the overall management of data resources in an organization. Database administrators are responsible for physical database design and for managing technical issues in the database environment.

8. **System Developers** are persons such as systems analysts and programmers who design new application programs. System developers often use CASE tools for system requirements analysis and program design.
9. **End Users** are persons throughout the organization who add, delete, and modify data in the database and who request or receive information from it. All user interactions with the database must be routed through the DBMS.

Activities/Assessment

1. Make your own infographic that differentiate data from information consider the example provided (figure 1.1)
2. List down 10 entities and identify their attributes.
3. Upon listing down the 10 entities. Provide their metadata.
4. Based on Figure 1.4 provide your analysis on their relationships.



LESSON 3

DESIGNING AND IMPLEMENTING DATABASE

Lesson Outcomes

At the end of the lesson, students will be able to:

1. Create and manage databases
2. Create and manage tables

Course Materials

Introduction

There several languages used in querying a database, either for commercial use or for free. However, SQL is the most widely used query language. It can perform more than as a “query language”. In addition, it can define the structure of the data, modify data in the database, and specify security constraints. This lesson presents the overview of SQL, process of creating and managing database, the process of creating and managing tables and the integrity constraints.

Installation Process

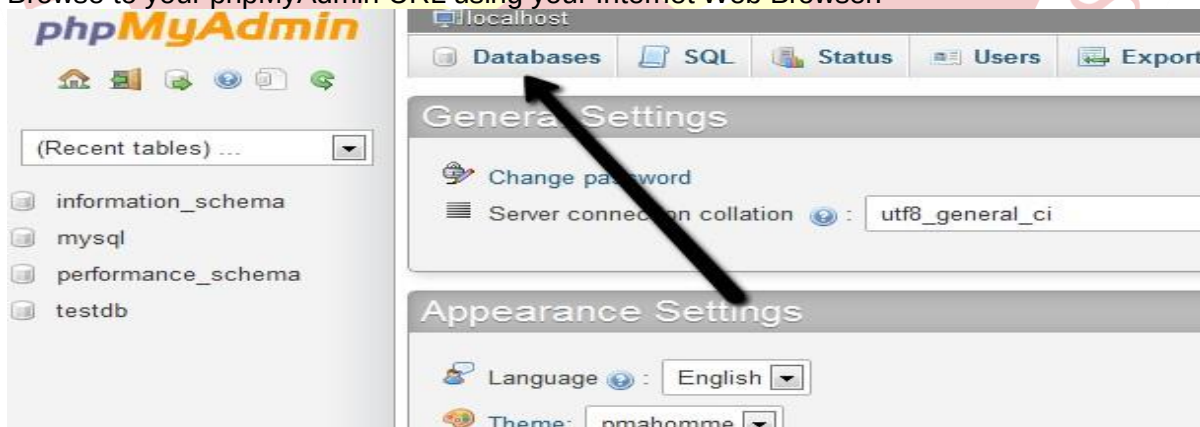
For the entire course the student will utilize MySQL. The steps for installing the software differs in different platforms. Visit the www.dev-mysql.com as reference but other databases that supports SQL is allowed.

Creating and Managing Databases

Creating and manipulating databases allow the user to perform different operations in the databases. A user can create, drop and back up a database using queries or statements provided below.

Steps in creating a database

1. Browse to your phpMyAdmin URL using your Internet Web Browser.



2. From the main menu choose Databases.



3. In the create database field type in a name for your database. Leave the collation drop down box if you wish to use the default MySQL schema collation. Click **Create**.

Databases

Create database

mydatabase Collation

Create

4. Your database will now be visible on the right-hand side under the list of available databases. To setup a new user login to access this database, click on **Users** in the main menu. Choose the **Add User** option under the list of available server users.

phpMyAdmin

(Recent tables) ...

- information_schema
- mydatabase
- mysql
- performance_schema
- testdb

localhost

Databases SQL Status **Users** Export Import Settings Synchroni

Users overview

User	Host	Password	Global privileges	Grant	Action
☐ Any	%	--	USAGE	No	Edit Privileges Export
☐ Any	localhost	No	USAGE	No	Edit Privileges Export
☐ backup	localhost	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export
☐ root	127.0.0.1	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export
☐ root	:::1	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export
☐ root	localhost	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export

↑ Check All / Uncheck All

Add user

5. In the section titled **Login Information** - type in a **username**, **localhost** and a **password** in the fields as shown. Optionally you can press the **Generate** button to create a random password for you.
6. The section that relates to the user's GLOBAL privileges are privileges you want to assign to this user which apply to **ALL databases** on the server. It is recommended that you do **NOT** assign these permissions unless you know exactly what you are doing. It is far more secure to assign separate user logins to each piece of software or website that will require access to only a *particular database*. Therefore, leave this section **BLANK**, and then click the **Add User** button.

Global privileges (Check All / Uncheck All)

Note: MySQL privilege names are presented in English

Data

- ☐ SELECT
- ☐ INSERT
- ☐ UPDATE
- ☐ DELETE
- ☐ FILE

Structure

- ☐ CREATE
- ☐ ALTER
- ☐ INDEX
- ☐ DROP
- ☐ CREATE TEMPORARY TABLES
- ☐ SHOW VIEW
- ☐ CREATE ROUTINE
- ☐ ALTER ROUTINE
- ☐ EXECUTE
- ☐ CREATE VIEW
- ☐ EVENT
- ☐ TRIGGER

Administration

- ☐ GRANT
- ☐ SUPER
- ☐ PROCESS
- ☐ RELOAD
- ☐ SHUTDOWN
- ☐ SHOW DATABASES
- ☐ LOCK TABLES
- ☐ REFERENCES
- ☐ REPLICATION CLIENT
- ☐ REPLICATION SLAVE
- ☐ CREATE USER

These Permissions assign the user GLOBAL permissions for ALL databases on the server.

7. After the user is created, you can see it listed on the Users page. Click **Edit Privileges** to assign access to a specific database.

Users overview

User	Host	Password	Global privileges	Grant	Action
☐ Any	%	—	USAGE	No	Edit Privileges Export
☐ Any	localhost	No	USAGE	No	Edit Privileges Export
☐ backup	localhost	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export
☒ mydatabase_admin	localhost	Yes	USAGE	No	Edit Privileges Export
☐ root	127.0.0.1	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export
☐ root	::1	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export
☐ root	localhost	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export

Check All / Uncheck All

8. Once again leave the Global Privileges section **BLANK**. Scroll down to the section titled **Database-Specific Privileges**. Choose the **database** you want the user to be able to access from the list, and click **GO**.

Database-specific privileges

Database Privileges Grant Table-specific privileges Action

None

Add privileges on the following database: mydatabase

Go

9. Assign the permissions as shown to provide the user with access to the given database. The following permissions are recommended for compatibility with most modern web- based software applications.

Database-specific privileges (Check All / Uncheck All)

Note: MySQL privilege names are expressed in English

Data Structure Administration

☒ SELECT
☒ INSERT
☒ UPDATE
☒ DELETE

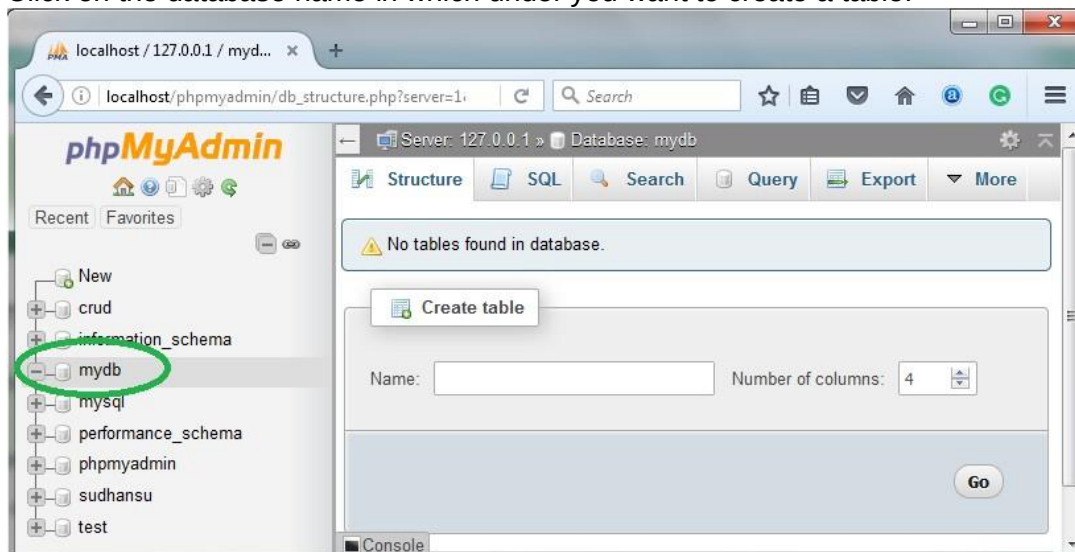
☒ CREATE
☒ ALTER
☒ INDEX
☒ DROP
☒ CREATE TEMPORARY TABLES
☐ SHOW VIEW
☐ CREATE ROUTINE
☐ ALTER ROUTINE
☐ EXECUTE
☐ CREATE VIEW
☐ EVENT
☐ TRIGGER

☐ GRANT
☒ LOCK TABLES
☐ REFERENCES

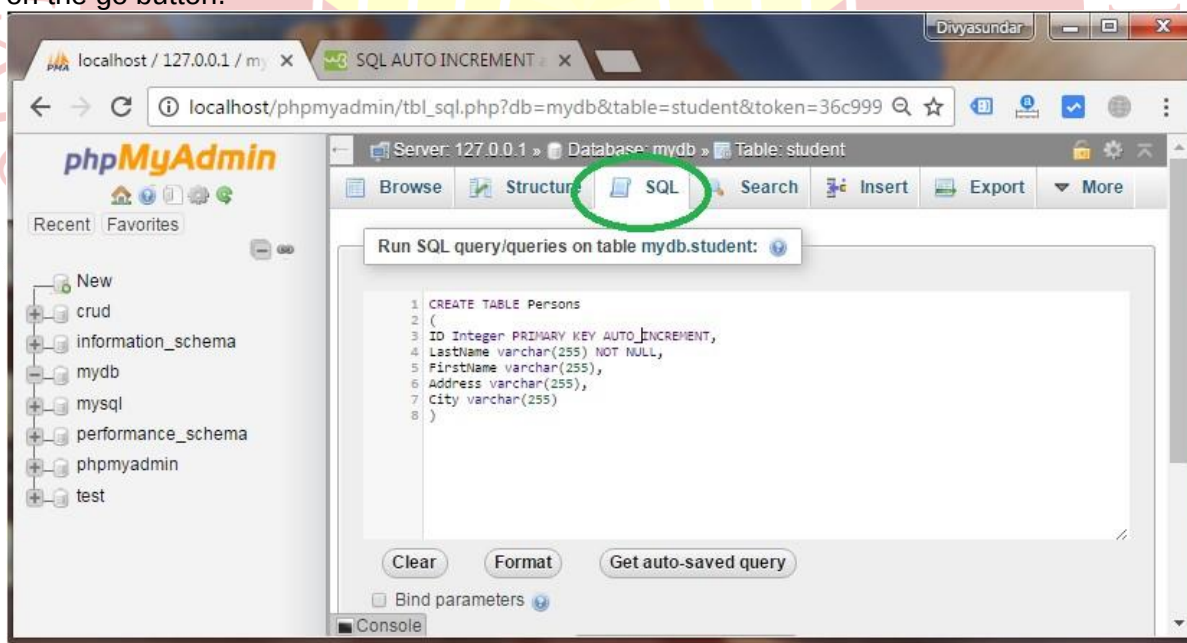
10. Click **GO** after selecting the relevant privileges.
11. Test your new user login by using it to login to phpMyAdmin. Use the account/user you just created, and you should only see a single database assigned to you.

Steps in creating table in a database

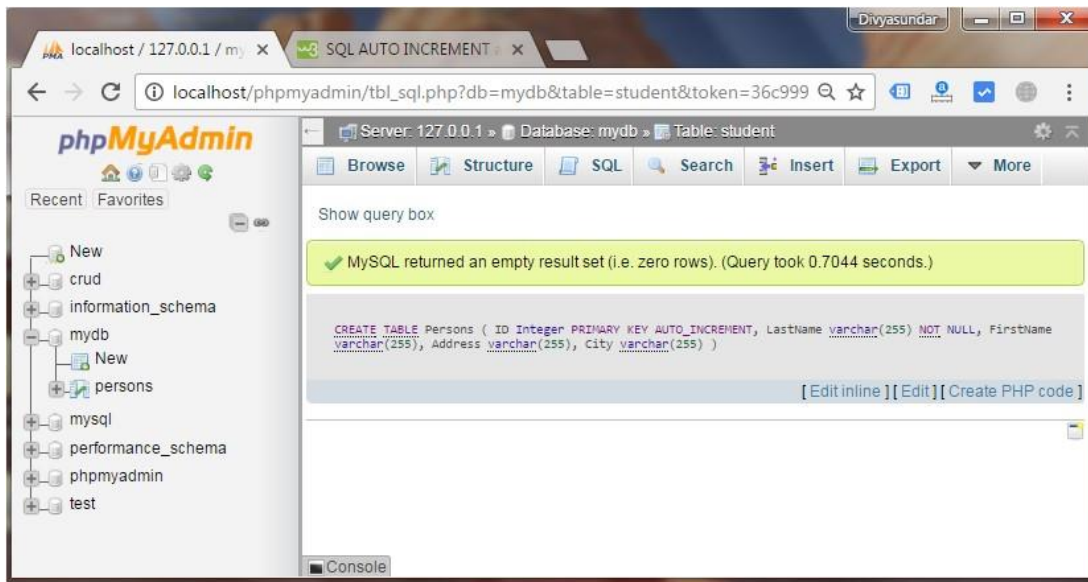
1. Click on the database name in which under you want to create a table.



2. Here you have two option to create table using **structure** and second one is using **SQL**. If you want to create table in structure option, then put your table name on the create table name field and choose columns and click on the go button. If you want to create a table by writing SQL Statements simply click on the SQL button on the page and write your query and click on the go button.



3. If your SQL Statement or table structure is correct you will be shown the page below.



Activities/Assessment

1. Create a database named 'db-sis'.
2. Once the database is created create a table named 'tbl-user' using CREATE TABLE Syntax with the following column:
 - user_id
 - uname
 - password

LESSON 4

THE OVERVIEW OF STRUCTURE QUERY LANGUAGE

Learning Outcomes

At the end of the lesson, students will be able to:

1. Explain the use and importance of integrity constraints
2. Determine how to enforce integrity constraints in program codes
3. Create a program code that will create databases/tables following the rules of integrity constraints
4. Ensure that table design is optimal i.e. proper data type/size is assigned to fields

Course Materials

IBM developed the original version of SQL, originally called Sequel, as part of the System R project in the early 1970s. The Sequel language has evolved since then, and its name has changed to SQL (Structured Query Language). Many products now support the SQL language. SQL has clearly established itself as the standard relational database language. In 1986, the American National Standards Institute (ANSI) and the International Organization for Standardization (ISO) published an SQL standard, called SQL-86. ANSI published an extended standard for SQL, SQL-89, in 1989. The next version of the standard was SQL-92 standard, followed by SQL:1999, SQL:2003, SQL:2006, and most recently SQL:2008. The bibliographic notes provide references to these standards.

The SQL language has several parts:

- **Data-definition language (DDL).** The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- **Data-manipulation language (DML).** The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- **Integrity.** The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
- **View definition.** The SQL DDL includes commands for defining views.
- **Transaction control.** SQL includes commands for specifying the beginning and ending of transactions.
- **Embedded SQL and dynamic SQL.** Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, and Java.
- **Authorization.** The SQL DDL includes commands for specifying access rights to relations and views.



The SQL CREATE DATABASE Statement

The **CREATE DATABASE** statement is used to create a new SQL database.

Syntax:

```
CREATE DATABASE databasename;
```

Example Query:

```
CREATE DATABASE test;
```

The statement above creates a database with name “test”. Once the database is created, you can check it in the list of databases with the following SQL command: SHOW DATABASES;

The SQL DROP DATABASE Statement

The **DROP DATABASE** statement is used to drop an existing SQL database.

Syntax:

```
DROP DATABASE databasename;
```

Example Query:

```
DROP DATABASE test;
```

The statement above shows an SQL statement to drop an existing database. Once a database is dropped, you can check it in the list of databases with the following SQL command: SHOW DATABASES;

The SQL BACKUP DATABASE Statement

The **BACKUP DATABASE** statement is used in SQL Server to create a full back up of an existing SQL database.

Syntax:

```
BACKUP DATABASE databasename  
TO DISK = 'filepath';
```

Example:

```
BACKUP DATABASE test  
TO DISK = 'D:\backups\test.bak';
```

The statement above shows an SQL statement to create a full back up of the existing database “test” to the D: disk.

The SQL BACKUP WITH DIFFERENTIAL Statement

A differential back up only backs up the parts of the database that have changed since the last full database backup.



Syntax:

```
BACKUP DATABASE databasename
TO DISK = 'filepath'
WITH DIFFERENTIAL;
```

Example:

```
BACKUP DATABASE testDB
TO DISK = 'D:\backups\test.bak'
WITH DIFFERENTIAL;
```

The statement above shows an SQL statement that creates a differential back up of the database “test”.

Creating and Managing Tables**The SQL CREATE TABLE statement**

We define an **SQL relation** by using the **create table** command.

Syntax:

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ....
);
```

The **column parameters** specify the names of the columns of the tables. The **datatype parameter** specifies the type of data the column can hold (e.g. varchar, integer, date, etc.).

The following example creates a table called “Persons” that contains five columns: PersonID, LastName, FirstName, Address, and City:

```
CREATE TABLE Persons (
    PersonID int,
    LastName varchar(255),
    FirstName varchar(255),
    Address varchar(255),
    City varchar(255)
);
```

The SQL DROP TABLE statement

The DROP TABLE statement is used to drop an existing table in a database.

Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE Persons;
```

The statement above drops an existing table “Persons”.



The SQL TRUNCATE TABLE

The TRUNCATE TABLE statement is used to delete the data inside a table, but not the table itself.

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE Persons;
```

The SQL ALTER TABLE Statement

The ALTER TABLE statement is used to add, delete, or modify columns in an existing table. The ALTER TABLE statement is also used to add and drop various constraints on an existing table.

A. **ALTER TABLE ADD COLUMN**– To add column to existing table.

Syntax:

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Example:

```
ALTER TABLE Customers  
ADD Email varchar(255);
```

B. **ALTER TABLE DROP COLUMN** – To drop

Syntax:

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Example:

```
ALTER TABLE Customers  
DROP COLUMN Email;
```

Constraints

The SQL Constraints

SQL supports a number of different integrity constraints. **SQL constraints** are used to specify rules for the data in a table. **Constraints** are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.

Constraints can be **column level** or **table level**. *Column level* constraints apply to a column, and *table level* constraints apply to the whole table.



The following constraints are commonly used in SQL:

- **NOT NULL** - Ensures that a column cannot have a NULL value
- **UNIQUE** - Ensures that all values in a column are different
- **PRIMARY KEY** - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- **FOREIGN KEY** - Uniquely identifies a row/record in another table
- **CHECK** - Ensures that all values in a column satisfies a specific condition
- **DEFAULT** - Sets a default value for a column when no value is specified
- **INDEX** - Used to create and retrieve data from the database very quickly

SQL NOT NULL Constraint

The not null constraint on an attribute specifies that the null value is not allowed for that attribute; in other words, the constraint excludes the null value from the domain of that attribute.

The following SQL ensures that the "ID", "LastName", and "FirstName" columns will NOT accept NULL values when the "Persons" table is created:

Example:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255) NOT NULL,
  Age int
);
```

SQL The UNIQUE Constraint

The unique constraint ensures that all values in a column are different. Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns. A PRIMARY KEY constraint automatically has a UNIQUE constraint. However, you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.

Example:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  UNIQUE (ID)
);
```

SQL PRIMARY KEY Constraints

The primary key must be unique and nonnull. It uniquely identifies the relation. It means that no tuple can have a null value for a primary-key attribute, and no two tuples in the relation can be equal on all the primary key attributes. It is used to link two tables together. The table containing the foreign key is called the child table, and the table containing the candidate key is called the referenced or parent table.



The example below specifies ID attribute as primary key.

Example:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  PRIMARY KEY (ID)
);
```

SQL FOREIGN KEY Constraints

The foreign key specification says that the values of attributes or any tuple in the relation must correspond to values of the primary key attributes of some tuple in relation.

The following SQL creates a FOREIGN KEY on the "PersonID" column when the "Orders" table is created:

Example:

```
CREATE TABLE Orders (
  OrderID int NOT NULL,
  OrderNumber int NOT NULL,
  PersonID int,
  PRIMARY KEY (OrderID),
  FOREIGN KEY (PersonID) REFERENCES Persons (PersonID)
);
```

SQL CHECK Constraint

The CHECK constraint is used to limit the value range that can be placed in a column. If you define a CHECK constraint on a single column it allows only certain values for this column. If you define a CHECK constraint on a table, it can limit the values in certain columns based on values in other columns in the row.

The CHECK constraint in example below ensures that you cannot have any person below 18 years:

Example:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  CHECK (Age>=18)
);
```

DEFAULT

The DEFAULT constraint is used to provide a default value for a column. The default value will be added to all new records IF no other value is specified.



Example:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  City varchar(255) DEFAULT 'Sandnes'
);
```

The DEFAULT constraint can also be used to insert system values, by using functions like GETDATE():

```
CREATE TABLE Orders (
  ID int NOT NULL,
  OrderNumber int NOT NULL,
  OrderDate date DEFAULT GETDATE()
);
```

SQL CREATE INDEX Statement

The CREATE INDEX statement is used to create indexes in tables. Indexes are used to retrieve data from the database very fast. The users cannot see the indexes; they are just used to speed up searches/queries.

Syntax:

```
CREATE INDEX index_name
ON table_name (column1, column2, ...);
```

Example:

```
CREATE INDEX idx_lastname
ON Persons (LastName);
```

UNIQUE INDEX creates a unique index on a table. Duplicate values are not allowed.

Activities/Assessment

1. Create your own database. You can choose your own database name.
2. Create your own table in the database and apply the Integrity constraints.
3. Back up the database to your specified disk.
4. Drop the database.

Submit your SQL Statements together with the database and the backup database.



LESSON 5 DATA MANIPULATING LANGUAGE

Lesson Outcomes

At the end of the lesson, students will be able to:

1. Utilize the different SQL Manipulating Language
2. Display data from multiple tables using JOIN, UNION, and sub queries
3. Implement the user of Restrict and Sort Data in the database
4. Identify the column functions and groupings of data
5. Identify the Scalar and Arithmetic Functions
6. Create and Manage Views

Course Materials

Introduction

SQL is a powerful query language that performs different operations in the database. This lesson will roughly cover these operations. It will start from defining and presenting examples of data manipulation language that includes INSERT, UPDATE and DELETE statements, displaying data from multiple tables such as JOIN, UNION, and Sub queries. The process of restricting and sorting data, column functions and grouping, scalar and arithmetic functions, and creating and managing views are also included in this lesson.

Using Data Manipulating Language

In order to manipulate data in relations in database queries are performed such as the INSERT statement that allows to insert new data in relations, UPDATE statement that allows to update existing data in relations, and the DELETE statement that allows to delete existing data in relations in the database.

INSERT INTO Statement

The **insert command** allows to load data into the relations.

Syntax:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland

Example:

The following SQL statement inserts a new record in the "Customers" table:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');
```



The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland
92	Cardinal	Tom Erichsen	Skagen 21	Stavanger	4006	Norway

Insert Data Only in Specified

The following SQL statement will insert a new record, but only insert data in the "CustomerName", "City", and "Country" columns (CustomerID will be updated automatically):

Example:

```
INSERT INTO Customers (CustomerName, City, Country)
VALUES ('Cardinal', 'Stavanger', 'Norway');
```

The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
89	White Clover Markets	Karl Jablonski	305 - 14th Ave. S. Suite 3B	Seattle	98128	USA
90	Wilman Kala	Matti Karttunen	Keskuskatu 45	Helsinki	21240	Finland
91	Wolski	Zbyszek	ul. Filtrowa 68	Walla	01-012	Poland
92	Cardinal	null	null	Stavanger	null	Norway

UPDATE Statement

The **update command** is used to modify tuples or existing records in a table. The WHERE clause specifies which record(s) that should be updated. If you omit the WHERE clause, all records in the table will be updated.

Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Example:

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;
```



The following SQL statement updates the first customer (CustomerID = 1) with a new contact person and a new city.

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Update Multiple Records

It is the WHERE clause that determines how many records that will be updated. The following SQL statement will update the ContactName to "Juan" for all records where country is "Mexico":

Syntax:

```
UPDATE Customers
SET ContactName='Juan'
WHERE Country='Mexico';
```

The selection from the "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Alfred Schmidt	Obere Str. 57	Frankfurt	12209	Germany
2	Ana Trujillo Emparedados y helados	Juan	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Juan	Mataderos 2312	México D.F.	05023	Mexico

DELETE Statement

The **delete command** is used to delete tuples in a relation. The WHERE clause specifies which record(s) should be deleted. If you omit the WHERE clause, all records in the table will be deleted.

Syntax:

```
DELETE FROM table_name WHERE condition;
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany



2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Example:

DELETE FROM Customers **WHERE** CustomerName='Alfreds Futterkiste';

The "Customers" table will now look like this:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Delete ALL Records

It is possible to delete all rows in a table without deleting the table. This means that the table structure, attributes, and indexes will be intact:

Syntax:

DELETE FROM *table_name*;

Activities/Assessment

Using the database provided, create statements for the following result sets.

1. Select all employees arranged by surname ascending, limit its results to 100.
2. Update the salary of employee with id 100001.
3. Change the department of employee 100001 to Customer Service.
4. Update the name of the department Production to Production I



LESSON 6 ADVANCED SQL

Learning Outcomes

At the end of the lesson, students will be able to:

1. Create program codes that will apply the DML commands
2. Test program codes to ensure they meet the requirements

Course Materials

Displaying data from multiple Tables

JOIN

A **JOIN** clause is used to combine rows from two or more tables, based on a related column between them.

Let's look at a selection from the "Orders" table:

OrderID	CustomerID	OrderDate
10308	2	1996-09-18
10309	37	1996-09-19
10310	77	1996-09-20

Then, look at a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Country
1	Alfreds Futterkiste	Maria Anders	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mexico

Notice that the "CustomerID" column in the "Orders" table refers to the "CustomerID" in the "Customers" table. The relationship between the two tables above is the "CustomerID" column.

Then, we can create the following SQL statement (that contains an INNER JOIN), that selects records that have matching values in both tables:

Example:

```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;
```

It will produce the following output:

OrderID	CustomerName	OrderDate
10308	Ana Trujillo Emparedados y helados	9/18/1996
10365	Antonio Moreno Taquería	11/27/1996
10383	Around the Horn	12/16/1996
10355	Around the Horn	11/15/1996
10278	Berglunds snabbköp	8/12/1996

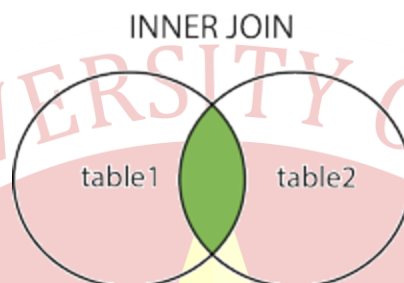


Inner Join

The **INNER JOIN** keyword selects records that have matching values in both tables.

Syntax:

```
SELECT column_name(s)
FROM table1
INNER JOIN table2 ON table1.column_name = table2.column_name;
```



Demo Database

Below is a selection from the "Orders" table:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10308	2	7	1996-09-18	3
10309	37	3	1996-09-19	1
10310	77	8	1996-09-20	2

And a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Example:

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;
```

The **INNER JOIN** keyword selects all rows from both tables as long as there is a match between the columns. If there are records in the "Orders" table that do not have matches in "Customers", these orders will not be shown.

JOIN Three Tables

The following SQL statement selects all orders with customer and shipper information:

Example:

```
SELECT Orders.OrderID, Customers.CustomerName, Shippers.ShipperName
FROM ((Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID)
INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID);
```

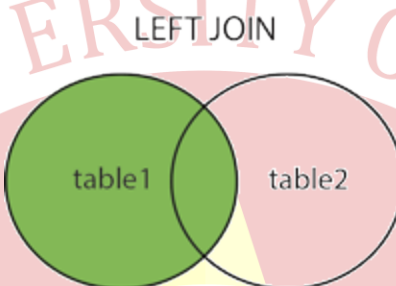


Left Join

The LEFT JOIN keyword returns all records from the left table (table1), and the matched records from the right table (table2). The result is NULL from the right side, if there is no match.

Syntax:

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2 ON table1.column_name = table2.column_name;
```



Demo Database

Below is a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

And a selection from the "Orders" table:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10308	2	7	1996-09-18	3
10309	37	3	1996-09-19	1
10310	77	8	1996-09-20	2

Example:

The following SQL statement will select all customers, and any orders they might have:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
ORDER BY Customers.CustomerName;
```

Right Join

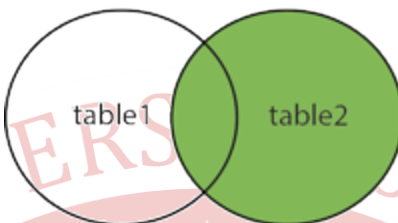
The RIGHT JOIN keyword returns all records from the right table (table2), and the matched records from the left table (table1). The result is NULL from the left side, when there is no match. In some databases RIGHT JOIN is called RIGHT OUTER JOIN.



Syntax:

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2 ON table1.column_name = table2.column_name;
```

RIGHT JOIN

**Demo Database**

Below is a selection from the "Orders" table:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10308	2	7	1996-09-18	3
10309	37	3	1996-09-19	1
10310	77	8	1996-09-20	2

And a selection from the "Employees" table:

EmployeeID	LastName	FirstName	BirthDate	Photo
1	Davolio	Nancy	12/8/1968	EmplD1.pic
2	Fuller	Andrew	2/19/1952	EmplD2.pic
3	Leverling	Janet	8/30/1963	EmplD3.pic

Example:

The following SQL statement will return all employees, and any orders they might have placed:

```
SELECT Orders.OrderID, Employees.LastName, Employees.FirstName
FROM Orders
RIGHT JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
ORDER BY Orders.OrderID;
```

Full Outer Join

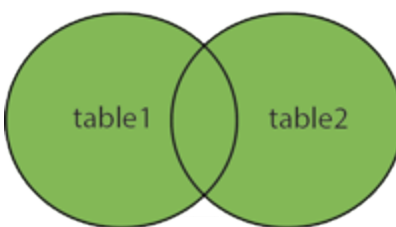
The FULL OUTER JOIN keyword returns all records when there is a match in either left (table1) or right (table2) table records. It preserves tuples in both relations. It can potentially return very large results.

Syntax:

```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2 ON table1.column_name = table2.column_name;
```



FULL OUTER JOIN

*Demo Database*

Below is a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

And a selection from the "Orders" table:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10308	2	7	1996-09-18	3
10309	37	3	1996-09-19	1
10310	77	8	1996-09-20	2

Example:

The following SQL statement selects all customers, and all orders:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
FULL OUTER JOIN Orders ON Customers.CustomerID=Orders.CustomerID
ORDER BY Customers.CustomerName;
```

Self Join

A self JOIN is a regular join, but the table is joined with itself.

Syntax:

```
SELECT column_name(s)
FROM table1 T1, table1 T2
WHERE condition;
```

Demo Database

Below is a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany

2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Example:

```
SELECT A.CustomerName AS CustomerName1, B.CustomerName AS CustomerName2,
A.City
FROM Customers A, Customers B
WHERE A.CustomerID <> B.CustomerID
AND A.City = B.City
ORDER BY A.City;
```

UNION

The UNION operator is used to combine the result-set of two or more SELECT statements.

- Each SELECT statement within UNION must have the same number of columns
- The columns must also have similar data types
- The columns in each SELECT statement must also be in the same order

Syntax:

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;
```

UNION ALL Syntax

```
SELECT column_name(s) FROM table1
UNION ALL
SELECT column_name(s) FROM table2;
```

Demo Database

Below is a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

And a selection from the "Suppliers" table:

SupplierID	SupplierName	ContactName	Address	City	PostalCode	Country
1	Exotic Liquid	Charlotte Cooper	49 Gilbert St.	London	EC1 4SD	UK
2	New Orleans Cajun Delights	Shelley Burke	P.O. Box 78934	New Orleans	70117	USA



3	Grandma Kelly's Homestead	Regina Murphy	707 Oxford Rd.	Ann Arbor	48104	USA
---	---------------------------	---------------	----------------	-----------	-------	-----

SQL UNION Example:

The following SQL statement returns the cities (only distinct values) from both the "Customers" and the "Suppliers" table:

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers
ORDER BY City;
```

SQL UNION ALL Example:

The following SQL statement returns the cities (duplicate values also) from both the "Customers" and the "Suppliers" table:

```
SELECT City FROM Customers
UNION ALL
SELECT City FROM Suppliers
ORDER BY City;
```

SQL UNION with WHERE Example:

The following SQL statement returns the German cities (only distinct values) from both the "Customers" and the "Suppliers" table:

```
SELECT City, Country FROM Customers
WHERE Country='Germany'
UNION
SELECT City, Country FROM Suppliers
WHERE Country='Germany'
ORDER BY City;
```

SQL UNION ALL with WHERE Example:

The following SQL statement returns the German cities (duplicate values also) from both the "Customers" and the "Suppliers" table:

```
SELECT City, Country FROM Customers
WHERE Country='Germany'
UNION ALL
SELECT City, Country FROM Suppliers
WHERE Country='Germany'
ORDER BY City;
```

Another UNION Example

The following SQL statement lists all customers and suppliers:

```
SELECT 'Customer' As Type, ContactName, City, Country
FROM Customers
UNION
SELECT 'Supplier', ContactName, City, Country
FROM Suppliers;
```



Sub Queries

SQL also provides the subquery technique, which involves placing an inner query (SELECT . . . FROM . . . WHERE) within a WHERE or HAVING clause of another (outer) query. The inner query provides a set of one or more values for the search condition of the outer query. Such queries are referred to as subqueries or nested subqueries. Subqueries can be nested multiple times. Subqueries are prime examples of why SQL is a set-oriented language.

Demo Database

Below is a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

And a selection from the "Orders" table:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10308	2	7	1996-09-18	3
10309	37	3	1996-09-19	1
10310	77	8	1996-09-20	2

Query: What are the name and address of the customer who placed order number 10308?

```
SELECT CustomerName, ContactName, Address, City, PostalCode, Country
FROM Customers, Orders
WHERE Customers.CustomerID = Orders.CustomerID AND OrderID = 10308;

SELECT CustomerName, ContactName, Address, City, PostalCode, Country
FROM Customers
WHERE Customers.CustomerID = (SELECT Orders.CustomerID FROM Orders WHERE
OrderID = 10308)
```

Notice that the subquery, shaded in blue and enclosed in parentheses, follows the form learned for constructing SQL queries and could stand on its own as an independent query. That is, the result of the subquery, as with any other query, is a set of rows—in this case, a set of CustomerID values. We know that only one value will be in the result. (There is only one CustomerID for the order with OrderID 10308.) To be safe, we can, and probably should, use the IN operator rather than = when writing subqueries. The subquery approach may be used for this query because we need to display data from only the table in the outer query. The value for OrderID does not appear in the query result; it is used as the selection criterion in the inner query. To include data from the subquery in the result, use the join technique, because data from a subquery cannot be included in the final results.

The query below produces the same output.

```
SELECT CustomerName
FROM Customers
WHERE CustomerID IN (SELECT DISTINCT CustomerID FROM Orders)
```



Restricting and Sorting Data

SELECT DISTINCT Statement

The SELECT DISTINCT statement is used to return only distinct (different) values. Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values.

Syntax

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Example

```
SELECT DISTINCT Country FROM Customers;
```

The following SQL statement lists the number of different (distinct) customer countries:

```
SELECT COUNT(DISTINCT Country) FROM Customers
```

SQL WHERE Clause

The WHERE clause is used to filter records. The WHERE clause is used to extract only those records that fulfill a specified condition.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico



3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Example:

```
SELECT * FROM Customers
WHERE Country='Mexico';
```

Text Fields and Numeric Fields

SQL requires single quotes around text values (most database systems will also allow double quotes). However, numeric fields should not be enclosed in quotes:

Example:

```
SELECT * FROM Customers
WHERE CustomerID=1;
```

Operators in WHERE Clause

Operator	Description
=	Equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

The BETWEEN Operator

The BETWEEN operator selects values within a given range. The values can be numbers, text, or dates. The BETWEEN operator is inclusive: begin and end values are included.

Syntax

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Demo Database

Below is a selection from the "Products" table in the Northwind sample database:

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	1	2	48 - 6 oz jars	22



5	Chef Anton's Gumbo Mix	1	2	36 boxes	21.35
---	------------------------	---	---	----------	-------

Example:

```
SELECT * FROM Products
WHERE Price BETWEEN 10 AND 20;
```

NOT BETWEEN Example

```
SELECT * FROM Products
WHERE Price NOT BETWEEN 10 AND 20;
```

The IN Operator

The IN operator allows you to specify multiple values in a WHERE clause. The IN operator is a shorthand for multiple OR conditions.

Syntax

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1, value2, ...);
```

Or

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (SELECT STATEMENT);
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Example

```
SELECT * FROM Customers
WHERE Country IN ('Germany', 'France', 'UK');
```

The following SQL statement selects all customers that are NOT located in "Germany", "France" or "UK":

```
SELECT * FROM Customers
WHERE Country NOT IN ('Germany', 'France', 'UK');
```



The following SQL statement selects all customers that are from the same countries as the suppliers:

```
SELECT * FROM Customers
WHERE Country IN (SELECT Country FROM Suppliers);
```

The LIKE Operator

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column. There are two wildcards used in conjunction with the LIKE operator:

- % - The percent sign represents zero, one, or multiple characters
- _ - The underscore represents a single character.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE columnN LIKE pattern;
```

Here are some examples showing different LIKE operators with '%' and '_' wildcards:

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE ' _r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a _% _%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Example Query:

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';
```



The LOGICAL Operators

The WHERE clause can be combined with AND, OR, and NOT operators. The AND and OR operators are used to filter records based on more than one condition:

- The AND operator displays a record if all the conditions separated by AND are TRUE.
- The OR operator displays a record if any of the conditions separated by OR is TRUE.

The NOT operator displays a record if the condition(s) is NOT TRUE.

AND Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...;
```

OR Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 OR condition2 OR condition3 ...;
```

NOT Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE NOT condition;
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

AND Example Query

```
SELECT * FROM Customers
WHERE Country='Germany' AND City='Berlin';
```

OR Example Query

```
SELECT * FROM Customers
WHERE City='Berlin' OR City='München';
```

NOT Example Query

```
SELECT * FROM Customers
WHERE NOT Country='Germany';
```



The ORDER BY Clause

The ORDER BY keyword is used to sort the result-set in ascending or descending order. The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.

Syntax

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

ORDER BY Example:

The following SQL statement selects all customers from the "Customers" table, sorted by the "Country" column:

```
SELECT * FROM Customers
ORDER BY Country;
```

ORDER BY DESC Example:

The following SQL statement selects all customers from the "Customers" table, sorted DESCENDING by the "Country" column:

```
SELECT * FROM Customers
ORDER BY Country DESC;
```

Column Functions and Grouping

A column function produces a single value for a group of rows. For example, if an SQL SELECT clause asks for the value of SUM(salary), it returns only one value, the sum.

COUNT Function

The COUNT() function finds the number of rows that satisfy the search condition or finds the number of distinct values in a particular column.

COUNT() Syntax

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```



AVG Function

The AVG function finds the average of the values in a particular column or a set of values that are derived from one or more columns. The column or expression that is summarized must contain numeric data.

AVG() Syntax

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

SUM Function

The SUM function finds the sum of the values in a particular column or a set of values that are derived from one or more columns. The column or expression that is added must contain numeric data.

SUM() Syntax

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

MAX() Function

The MAX function finds the value in a particular column or a set of value that are derived from one or more columns. MAX applies to all data type.

MAX() Syntax

```
SELECT MAX(column_name)
FROM table_name
WHERE condition;
```

MIN() Function

The MIN function finds the minimum value in a particular column or set of values that are derived from one or more columns. MIN applies to all data types.

MIN() Syntax

```
SELECT MIN(column_name)
FROM table_name
WHERE condition;
```

Demo Database

Below is a selection from the "Products" table in the Northwind sample database:

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35



COUNT() Example

```
SELECT COUNT(ProductID)
FROM Products;
```

AVG() Example

```
SELECT AVG(Price)
FROM Products;
```

SUM() Example

```
SELECT SUM(Quantity)
FROM OrderDetails;
```

MAX() Example

```
SELECT MAX(Price) AS LargestPrice
FROM Products;
```

MIN() Example

```
SELECT MIN(Price) AS SmallestPrice
FROM Products;
```

GROUP BY Statement

The GROUP BY statement groups rows that have the same values into summary rows, like “find the number of each customers in each country”.

The GROUP BY statement is often used with aggregate functions (COUNT, MAX, MIN, SUM, AVG) to group the result-set by one or more columns.

GROUP BY Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden



GROUP BY Example

The following SQL statement lists the number of customers in each country:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;
```

The following SQL statement lists the number of customers in each country, sorted high to low:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

GROUP By with JOIN Example

The following SQL statement lists the number of orders sent by each shipper:

Demo Database

Below is a selection from the "Orders" table in the Northwind sample database:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10248	90	5	1996-07-04	3
10249	81	6	1996-07-05	1
10250	34	4	1996-07-08	2

And a selection from the "Shippers" table:

ShipperID	ShipperName
1	Speedy Express
2	United Package
3	Federal Shipping

```
SELECT Shippers.ShipperName, COUNT(Orders.OrderID) AS NumberOfOrders FROM Orders
LEFT JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID
GROUP BY ShipperName;
```

Scalar and Arithmetic Function

Scalar Functions

Scalar Functions are based on user input, these too returns single value.

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico



4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

1. UCASE() Function - It converts the value of a field to uppercase.

Syntax:

```
SELECT UCASE(column_name) AS some_name FROM table_name;
```

Example Query

```
SELECT UCASE(CustomerName) AS UppercaseCustomerName
FROM Customers;
```

2. LCASE() Function - It converts the value of a field to lowercase.

Syntax:

```
SELECT LCASE(column_name) AS some_name FROM table_name;
```

Example Query

```
SELECT LCASE(CustomerName) AS LowercaseCustomerName FROM customers;
```

3. MID() – The MID() function extracts texts from the text field.

Syntax:

```
SELECT MID(column_name, start, length) AS some_name FROM table_name;
```

Example Query

```
SELECT MID(CustomerName, 1, 5) FROM Customers
```

4. LEN() Function – It returns the length of the value in a text field.

Syntax:

```
SELECT LENGTH(column_name) FROM tablename;
```

Example Query:

```
SELECT LENGTH(CustomerName) FROM Customers;
```

5. ROUND() – It is used to round a numeric field to the number of decimals specified.

Syntax:

```
SELECT ROUND(column_name, decimals) FROM products;
```

Example Query:

```
SELECT ROUND(price, 0) FROM products
```

7. NOW() Function – It is used to return the current system date and time.

Syntax:

```
SELECT CustomerName, NOW() AS DateTime FROM Customers
```

Example Query:

```
SELECT CustomerName, NOW() AS DateTime FROM Customers;
```

8. FORMAT() – It is used to format how a field is to be displayed

Syntax:

```
SELECT FORMAT(column_name, format) FROM table_name;
```

Example Query:

```
SELECT CustomerName, FORMAT(Now(), 'YYYY-MM-DD') AS Date FROM Customers;
```



Arithmetic Functions

A mathematical function executes a mathematical operation usually based on input values that are provided as arguments and return a numeric value as the result of the operation. Mathematical functions operate on numeric data such as decimal, integer, float, real, smallint, and tinyint.

Activities/Assessments

Refer to the Demo tables

1. Create an SQL statement that will display the ONLY items under CategoryID.
2. Create an SQL statement that will transform the CustomerName in uppercase.
3. Group the tbl_customer based on the country
4. Using the SQL Statement in #3, Arrange the CustomerName in descending order.



LESSON 7 SUBQUERIES

Learning Outcomes

At the end of the lesson, students will be able to:

1. Analyze problems to determine when to use a subquery
2. Create program codes for problems requiring subquery
3. Test program codes to ensure they meet the requirements

Course Materials

Subqueries

A Subquery or **Inner query** or a **Nested query** is a query within another SQL query and embedded within the WHERE clause. A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved. Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.

Rules for subqueries:

1. Subqueries must be **enclosed within parentheses**.
2. A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
3. An ORDER BY command cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY command can be used to perform the same function as the ORDER BY in a subquery.
4. Subqueries that return more than one row can only be used with multiple value operators such as the IN operator.
5. The SELECT list cannot include any references to values that evaluate to a BLOB, ARRAY, CLOB, or NCLOB.
6. A subquery cannot be immediately enclosed in a set function.
7. The BETWEEN operator cannot be used with a subquery. However, the BETWEEN operator can be used within the subquery.

Consider the following table:

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	35	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00



Subqueries are most frequently used with the SELECT statement.

```
SELECT column_name [, column_name ]
FROM table1 [, table2 ]
WHERE column_name OPERATOR
      (SELECT column_name [, column_name ]
        FROM table1 [, table2 ]
        [WHERE])
```

Example:

```
SELECT *
FROM CUSTOMERS
WHERE ID IN (
  SELECT ID
  FROM CUSTOMERS
  WHERE SALARY > 4500);
```

ID	NAME	AGE	ADDRESS	SALARY
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
7	Muffy	24	Indore	10000.00

Subqueries also can be used with **INSERT** statements. The INSERT statement uses the data returned from the subquery to insert into another table. The selected data in the subquery can be modified with any of the character, date or number functions.

```
INSERT INTO table_name [ (column1 [, column2 ]) ]
SELECT [ *|column1 [, column2 ]
FROM table1 [, table2 ]
[ WHERE VALUE OPERATOR ]
```

Consider a table CUSTOMERS_BKP with similar structure as CUSTOMERS table. Now to copy the complete CUSTOMERS table into the CUSTOMERS_BKP table, you can use the following syntax.

```
INSERT INTO CUSTOMERS_BKP
SELECT * FROM CUSTOMERS
WHERE ID IN (
  SELECT ID
  FROM CUSTOMERS
);
```

The subquery can be used in conjunction with the **UPDATE** statement. Either single or multiple columns in a table can be updated when using a subquery with the UPDATE statement.

```
UPDATE table
SET column_name = new_value
[ WHERE OPERATOR [ VALUE ]
  (SELECT COLUMN_NAME
   FROM TABLE_NAME)
  [ WHERE) ]
```



Assuming, we have CUSTOMERS_BKP table available which is backup of CUSTOMERS table. The following example updates SALARY by 0.25 times in the CUSTOMERS table for all the customers whose AGE is greater than or equal to 27.

```
UPDATE CUSTOMERS
SET SALARY = SALARY * 0.25
WHERE AGE IN (SELECT AGE FROM CUSTOMERS_BKP
              WHERE AGE >= 27 );
```

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	35	Ahmedabad	125.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	2125.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

The subquery can be used in conjunction with the **DELETE** statement like with any other statements mentioned above. The basic syntax is as follows.

```
DELETE FROM TABLE_NAME
[ WHERE OPERATOR [ VALUE ]
  (SELECT COLUMN_NAME
   FROM TABLE_NAME)
[ WHERE ) ]
```

Assuming, we have a CUSTOMERS_BKP table available which is a backup of the CUSTOMERS table. The following example deletes the records from the CUSTOMERS table for all the customers whose AGE is greater than or equal to 27.

```
DELETE FROM CUSTOMERS
WHERE AGE IN (SELECT AGE
              FROM CUSTOMERS_BKP
              WHERE AGE >= 27 );
```

ID	NAME	AGE	ADDRESS	SALARY
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Activities / Assessment

Using the database provided, create SQL statements for the following result set.

1. Select the total number of employees for each country using subqueries.



LESSON 8

SQL VIEWS

Learning Outcomes

At the end of the lesson, students will be able to:

1. Determine when to use VIEWS and what are its advantages
2. Determine when to use Stored Procedures and what are its advantages
3. Create program codes for problems requiring subquery
4. Test program codes to ensure they meet the requirements

Course Materials

Views

A database view is a **searchable object** in a database that is defined by a query. Though a view doesn't store data, some refer to a views as "**virtual tables**," you can query a view like you can a table. A view can combine data from two or more table, using joins, and also just contain a subset of information. This makes them convenient to abstract, or hide, complicated queries.

A views is created from a query using the **CREATE VIEW** command. In the example below we are creating a PopularBooks view based of a query which selects all Books that have the IsPopular field checked.

```
CREATE VIEW PopularBooks AS
SELECT ISBN, Title, Author, PublishDate
FROM Books
WHERE IsPopular = 1;
```

Once a view is created you can use them as you would any table in a SELECT statement. For example, to list all the popular book titles ordered by the author you could write:

```
SELECT Author, Title
FROM PopularBooks
ORDER BY Author;
```

There are many benefits to using views. Listed below are some of them:

- **Enforce Business Rules** – Use views to define business rules, such as when an item is active, or what is meant by "popular." By placing complicated or misunderstood business logic into the view, you can be sure to present a unified portrayal of the data. These increases use and quality.
- **Consistency** – Simplify complicated query logic and calculations by hiding it behind the view's definition. Once defined they calculations are reference from the view rather than being restated in separate queries. This makes for fewer mistakes and easier maintenance of code.
- **Security** – Restrict access to a table yet allow users to access non-confidential data via views. For example, you can restrict access to the employee table, that contains social security numbers, but allow access to a view containing the name and phone number.
- **Simplicity** – Databases with many tables possess complex relationships, which can be difficult to navigate if you are not comfortable using Joins. Use views to provide a "flattened"



view of the database for reporting or ad-hoc queries.

- **Space** – Views take up very little space, as the data is stored once in the source table. Some DBMS all you to create an index on a view, so in some cases views do take up more space than the definition.

Views also has its drawbacks:

- **Performance** – What may seem like a simple query against a view could turn out to be a hugely complex job for the database engine. That is because each time a view is referenced, the query used to define it, is rerun.
- **Modifications** – Not all views support INSERT, UPDATE, or DELETE operations. In general, in order to support these operations, the primary key and required fields must be present in the view. Complex multi-table views are generally read-only.

Activities / Assessment

Use the database provided:

1. Creates a view that shows all customers from Mexico
2. Create a SQL Statement that will show Customers from Mexico



LESSON 9 STORE PROCEDURE

Learning Outcomes

At the end of the lesson, students will be able to:

1. Determine when to use Stored Procedures and what are its advantages
2. Create program codes for problems requiring subquery
3. Test program codes to ensure they meet the requirements

Course Materials

A stored procedure is **a group of one or more database statements** stored in the database's data dictionary. SQL stored procedures can be called from T-SQL using an execute command that can be called from a program outside the database server such as a webserver or a client program.

```
CREATE PROCEDURE GETEMP ()  
SELECT  
    employeeID,  
    firstname,  
    lastname,  
    reg_date  
FROM  
    tblemployee  
ORDER BY lastname;
```

Once you save the stored procedure, you can invoke it by using the `CALL` statement:

```
CALL GETEMP ();
```

The first time you invoke a stored procedure, MySQL looks up for the name in the database catalog, compiles the stored procedure's code, place it in a memory area known as a cache, and execute the stored procedure. If you invoke the same stored procedure in the same session again, MySQL just executes the stored procedure from the cache without having to recompile it.

A stored procedure can have parameters so you can pass values to it and get the result back. For example, you can have a stored procedure that returns customers by country and city. In this case, the country and city are parameters of the stored procedure.




```
CREATE PROCEDURE GetOfficeByCountry(
    IN countryName VARCHAR(255)
)
BEGIN
    SELECT *
    FROM offices
    WHERE country = countryName;
END;
```

In this example, the `countryName` is the `IN` parameter of the stored procedure. Suppose that you want to find offices locating in the USA, you need to pass an argument (USA) to the stored procedure as shown in the following query:

```
CALL GetOfficeByCountry('USA');
```

	officeCode	city	phone	addressLine1	addressLine2	state	country	postalCode	territory
▶	1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA	USA	94080	NA
	2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA	USA	02107	NA
	3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY	USA	10022	NA

```
CREATE PROCEDURE GetOrderCountByStatus (
    IN orderStatus VARCHAR(25),
    OUT total INT
)
BEGIN
    SELECT COUNT(orderNumber)
    INTO total
    FROM orders
    WHERE status = orderStatus;
END
```

You can also have a stored procedure with OUT parameters. To find the number of orders that already shipped, you call `GetOrderCountByStatus` and pass the order status as of Shipped, and also pass a session variable (`@total`) to receive the return value.

```
CALL GetOrderCountByStatus('Shipped',@total);
SELECT @total;
```

	@total
▶	303



A stored procedure may contain control flow statements such as IF, CASE, and LOOP that allow you to implement the code in the procedural way. A stored procedure can call other stored procedures or stored functions, which allows you to modularize your code.

Advantages of stored procedures:

- **Reduce network traffic** - Stored procedures help reduce the network traffic between applications and MySQL Server. Because instead of sending multiple lengthy SQL statements, applications must send only the name and parameters of stored procedures.
- **Centralize business logic in the database** - You can use the stored procedures to implement business logic that is reusable by multiple applications. The stored procedures help reduce the efforts of duplicating the same logic in many applications and make your database more consistent.
- **Make database more secure** - The database administrator can grant appropriate privileges to applications that only access specific stored procedures without giving any privileges on the underlying tables.

Disadvantages of stored procedures

- **Resource usages** - If you use many stored procedures, the memory usage of every connection will increase substantially. Besides, overusing a large number of logical operations in the stored procedures will increase the CPU usage because the MySQL is not well-designed for logical operations.
- **Troubleshooting** - It's difficult to debug stored procedures. Unfortunately, MySQL does not provide any facilities to debug stored procedures like other enterprise database products such as Oracle and SQL Server.
- **Maintenances** - Developing and maintaining stored procedures often requires a specialized skill set that not all application developers possess. This may lead to problems in both application development and maintenance.

Activities/Assessment 1

1. What is Store Procedure?
2. What statement is used to invoke the stored procedure?
3. Using the demo tables, create a stored procedure that will:
 - a) Select all the employee in ascending order.



LESSON 10

TRIGGERS

Lesson Outcomes:

At the end of the lesson, students will be able to:

1. Determine when to use Stored Procedures and what are its advantages
2. Create program codes for problems requiring subquery
3. Test program codes to ensure they meet the requirements

Course Materials

Triggers

A trigger is a **set of actions that are run automatically** when a specified change operation (SQL INSERT, UPDATE, or DELETE statement) is performed on a specified table. Triggers are useful for tasks such as enforcing business rules, validating input data, and keeping an audit trail.

Benefits of using triggers in business:

- Faster application development. Because the database stores triggers, you do not have to code the trigger actions into each database application.
- Global enforcement of business rules. Define a trigger once and then reuse it for any application that uses the database.
- Easier maintenance. If a business policy changes, you need to change only the corresponding trigger program instead of each application program.
- Improve performance in client/server environment. All rules run on the server before the result returns.

A trigger is a named database object that is associated with a table, and it activates when a particular event (e.g. an insert, update or delete) occurs for the table. The statement CREATE TRIGGER creates a new trigger in MySQL. Here is the syntax:

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE }
ON table_name FOR EACH ROW
trigger_body;
```

There are 6 different types of triggers in MySQL:

1. **Before Update Trigger** - As the name implies, it is a trigger which enacts before an update is invoked. If we write an update statement, then the actions of the trigger will be performed before the update is implemented.
2. **After Update Trigger** - As the name implies, this trigger is invoked after an updating occurs. (i.e., it gets implemented after an update statement is executed.).
3. **Before Insert Trigger** - As the name implies, this trigger is invoked before an insert, or before an insert statement is executed.
4. **After Insert Trigger** - As the name implies, this trigger gets invoked after an insert is implemented.



5. **Before Delete Trigger** - As the name implies, this trigger is invoked before a delete occurs, or before deletion statement is implemented.
6. **After Delete Trigger** - As the name implies, this trigger is invoked after a delete occurs, or after a delete operation is implemented.

Additional Examples can be found here: <https://www.geeksforgeeks.org/different-types-of-mysql-triggers-with-examples/>

Activities / Assessment

Using the database provided, create triggers for the CRUD operation of the following:

1. Employees table should be reflected on Customer table.

PROJECT

Group yourself with at least 3 members. Create a dynamic web application. You may choose the following topic. Presentation of your project focusing on the database structure and the implementation of SQL Statements must be highlighted.

1. Student Information System (SIS)
2. Online Document Request
3. Document Tracking System
4. e-commerce
5. Learning Management System
6. University Website
7. Newspaper Website



GRADING SYSTEM**MIDTERMS****Class Standing**Portfolio/e-portfolio,
Activities/Assessment**60%**Practical Exam,
Project**40%****Midterm Grade****100%****TOTAL:****FINALS****Class Standing**Portfolio/e-portfolio,
Activities/Assessment.**60%**Practical Exam,
Project**40%****Final Grade****100%**

$$(MG + FG) / 2 = FR (100\%)$$

REFERENCES

Fundamentals of Database Management Systems Second Edition by Mark L. Gillenson
Database Systems by Thomas Connolly and Carolyn Begg



APPENDIX

tbl_Customer

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden
6	Blauer See Delikatessen	Hanna Moos	Forsterstr. 57	Mannheim	68306	Germany
7	Blondel père et fils	Frédérique Citeaux	24, place Kléber	Strasbourg	67000	France
8	Bólido Comidas preparadas	Martín Sommer	C/ Araquil, 67	Madrid	28023	Spain
9	Bon app'	Laurence Lebihans	12, rue des Bouchers	Marseille	13008	France
10	Bottom-Dollar Marketse	Elizabeth Lincoln	23 Tsawassen Blvd.	Tsawassen	T2F 8M4	Canada

tbl_employees

EmployeeID	LastName	FirstName	BirthDate	Photo	Notes
1	Davolio	Nancy	1968-12-08	EmpID1.pic	Education includes a BA in psychology from Colorado State University. She also completed (The Art of the Cold Call). Nancy is a member of 'Toastmasters International'.
2	Fuller	Andrew	1952-02-19	EmpID2.pic	Andrew received his BTS commercial and a Ph.D. in international marketing from the University of Dallas. He is fluent in French and Italian and reads German. He joined the company as a sales representative, was promoted to sales manager and was then named vice president of sales. Andrew is a member of the Sales Management Roundtable, the Seattle Chamber of Commerce, and the Pacific Rim Importers Association.
3	Leverling	Janet	1963-08-30	EmpID3.pic	Janet has a BS degree in chemistry from Boston College). She has also completed a certificate program in food retailing management. Janet was hired as a sales associate and was promoted to sales representative.
4	Peacock	Margaret	1958-09-19	EmpID4.pic	Margaret holds a BA in English literature from Concordia College and an MA from the American Institute of Culinary Arts.



					She was temporarily assigned to the London office before returning to her permanent post in Seattle.
5	Buchanan	Steven	1955-03-04	EmpID5.pic	Steven Buchanan graduated from St. Andrews University, Scotland, with a BSC degree. Upon joining the company as a sales representative, he spent 6 months in an orientation program at the Seattle office and then returned to his permanent post in London, where he was promoted to sales manager. Mr. Buchanan has completed the courses 'Successful Telemarketing' and 'International Sales Management'. He is fluent in French.

tbl_products

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
1	Chais	1	1	10 boxes x 20 bags	18
2	Chang	1	1	24 - 12 oz bottles	19
3	Aniseed Syrup	1	2	12 - 550 ml bottles	10
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22
5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35
6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25
7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	30
8	Northwoods Cranberry Sauce	3	2	12 - 12 oz jars	40
9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97
10	Ikura	4	8	12 - 200 ml jars	31

tbl_categories

CategoryID	CategoryName	Description
1	Beverages	Soft drinks, coffees, teas, beers, and ales
2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings
3	Confections	Desserts, candies, and sweet breads
4	Dairy Products	Cheeses
5	Grains/Cereals	Breads, crackers, pasta, and cereal
6	Meat/Poultry	Prepared meats
7	Produce	Dried fruit and bean curd
8	Seafood	Seaweed and fish



tbl_suppliers

SupplierID	SupplierName	ContactName	Address	City	PostalCode	Country	Phone
1	Exotic Liquid	Charlotte Cooper	49 Gilbert St.	Londona	EC1 4SD	UK	(171) 555-2222
2	New Orleans Cajun Delights	Shelley Burke	P.O. Box 78934	New Orleans	70117	USA	(100) 555-4822
3	Grandma Kelly's Homestead	Regina Murphy	707 Oxford Rd.	Ann Arbor	48104	USA	(313) 555-5735
4	Tokyo Traders	Yoshi Nagase	9-8 Sekimai Musashino-shi	Tokyo	100	Japan	(03) 3555-5011
5	Cooperativa de Quesos 'Las Cabras'	Antonio del Valle Saavedra	Calle del Rosal 4	Oviedo	33007	Spain	(98) 598 76 54
6	Mayumi's	Mayumi Ohno	92 Setsuko Chuo-ku	Osaka	545	Japan	(06) 431-7877
7	Pavlova, Ltd.	Ian Devling	74 Rose St. Moonie Ponds	Melbourne	3058	Australia	(03) 444-2343
8	Specialty Biscuits, Ltd.	Peter Wilson	29 King's Way	Manchester	M14 GSD	UK	(161) 555-4448
9	PB Knäckebröd AB	Lars Peterson	Kaloadagatan 13	Göteborg	S-345 67	Sweden	031-987 65 43
10	Refrescos Americanas LTDA	Carlos Diaz	Av. das Americanas 12.890	São Paulo	5442	Brazil	(11) 555 4640

